

StartHub Capital — Functionality & Flow Report

A two-sided platform that matches startups with investors. Users build rich profiles, discover the opposite side, send connection requests, chat in real time after connecting, and receive notifications.

1. High-Level Architecture

- **Monorepo with two apps:**
 - **client/:** React 18 + Vite + Tailwind SPA on port 3000, proxies /api to `http://localhost:5000`.
 - **server/:** Express 5 (ESM) + PostgreSQL (Supabase-hosted via `pg.Pool`) + Socket.io + Supabase Storage on port 5000.
- **Storage:**
 - Postgres tables managed by versioned SQL files under `supabase/migrations/`.
 - Binary assets in 4 Supabase buckets: `startup_logo`, `startup_documents`, `investor_photos`, `message-attachments`.
- **Boot Sequence (server/server.js):**
 - Express middleware (`helmet`, `CORS *`, `JSON/urlencoded`, `static /uploads`).
 - Health check `GET /api/health` (round-trips a `SELECT NOW()` to Postgres).
 - Mount 11 route groups (`auth`, `account`, `settings`, `startup profile`, `investor profile`, `investor search`, `uploads`, `messages`, `search`, `connections`, `notifications`).
 - Wrap Express in `http.createServer`, attach Socket.io with `CORS *`, call `initSocketServer(io)`.
 - Register a node-cron job at `0 2 * * * Asia/Colombo` that calls `deleteStaleUnverifiedUsers()`.

2. Authentication & Session Flow

Implemented in `server/controllers/authController.js` and consumed by `client/src/context/AuthContext.jsx`.

Token Type	Details
Access Token	JWT signed with JWT_SECRET, 15-minute lifetime, carries {userId, email, userType}.
Refresh Token	64-byte random hex string stored in the sessions table with device info, IP, and expires_at (24h or 30 days).

Registration & Login Chain

- **POST /api/auth/register:** Hashes password with bcrypt (10 rounds), inserts user with email_verified = false and a JWT verification token, emails link.
- **GET /api/auth/verify-email:** Verifies token, sets email_verified = true, clears tokens, redirects to frontend login.
- **POST /api/auth/login:** Checks account lock, compares password, rejects unverified emails, issues tokens, writes session.
- **POST /api/auth/token:** Exchanges refresh token for a new access token (lifetime NOT extended).
- **POST /api/auth/forgot-password & reset-password:** Manages password reset tokens, forces re-login everywhere upon reset.
- **POST /api/auth/logout:** Deletes one session. **logout-all** deletes all sessions.

3. Onboarding & Profile Flow

- **Two Parallel Worlds:** Startup and Investor profiles have their own controllers and repositories but reuse some logic.
- **Endpoints:** POST /, GET /me, GET /completion, PUT /:id, GET /:id.
- **Upload Pipeline:** Multer saves to tmp disk → pushed to Supabase bucket → public URL persisted → tmp file unlinked.
- **Privacy Gating:** GET /:id checks privacy_settings and connections to restrict fields.

4. Discovery / Search Flow

- **Pagination & Filters:** Supported for startups and investors (e.g., industry, stage, revenue).

- **Full-text Search:** Backed by Postgres FTS indexes.
- **Connection Status:** Each row is annotated with `connection_status` (self, not_connected, pending, accepted) in batch to avoid N+1 queries.

5. Connections Flow

- **Invariant:** One investor + one startup. Same-type pairs rejected.
- **State Machine:** pending → accepted (normalized from connected).
- **Key Endpoints:** POST `/api/connections` (create), PATCH (accept/decline), DELETE (remove).

6. Messaging Flow (REST + WebSocket)

- **REST (Writes):** POST `/api/messages` (text/attachments, gated by accepted connection), GET `/api/messages/conversations`.
- **WebSocket (Pushes):** Uses Socket.io. Tracks online users via memory Map. Emits `user:online`, `typing:start`, `message:new`, and `message:read:receipt`.

7. Notifications Flow

- **Unified Feed:** Aggregates pending connection requests and unread conversations.
- **Deduplication:** Deterministic keys allow client-side dedupe and read tracking via `user_notification_reads`.

8. Account & Settings Flow

- **Account:** PUT `/email` (with verify step), PUT `/password`, DELETE `/`.
- **Settings:** Privacy and notification settings toggles.

9. End-to-End User Journey (Startup)

1. Sign up → Verify Email → Login.
2. Bounced to onboarding if no profile exists.
3. Submit onboarding (multipart data) → Redirected to dashboard.
4. Discover investors → Send connect request.
5. Investor accepts → Chat via WebSocket/REST.

10. Cross-Cutting Concerns

- **Security:** helmet, CORS (*), bcrypt, JWT routes.
- **Rate Limiting:** Installed but currently a stub.
- **Cron:** Daily 02:00 Asia/Colombo cleanup of unverified users.
- **Migrations:** 18 SQL files, including recent FTS and schema refinements.

Notable Quirks

Quirk	Impact / Note
Typo in Route File	server/routes/connectons.js is missing an 'i'. Must update import in server.js if renamed.
Duplicate Route Mount	/api/startups is mounted twice (at /startups and /api/startups).
Circular-ish Import	clearOnboardingCache is called from AuthContext but defined in App.jsx. Fragile coupling.
Swallowed Errors	OnboardingGuard swallows network errors to favor availability over strictness.
Missing Migration Prefix	create_password_reset_tokens.sql lacks a timestamp prefix and must be manually applied.